ADL HW2 Report

Li-Jung Chen

Q1: LLM Tuning

- Describe:
  - I use all the train.json data as training dataset, which contains 10,000 examples.
  - I randomly pick 4 examples from training dataset for each instruction as their output examples, shown in Figure 1.
  - The hyper-parameters are as Figure 3.

- Show your performance:
  - The final performance of my model on the public testing set, measured by evaluation loss, is 2.48116.
  - The learning curve on the public testing set is as the orange line in Figure 2.

```python
def get_prompt(instruction) -> str:
    examples = train_data.shuffle(seed=random.randint(0, len(train_data))).select(range(4))
    examples = [x for x in examples]

    examples_text = ""
    for i, ex in enumerate(examples, 1):
        examples_text += (
            f"範例{i}：\n"
            f"指令：{ex['instruction']}\n"
            f"輸出：{ex['output']}\n\n"
        )

    prompt = (
            "請根據以下範例進行文言文與白話文互譯：\n\n"
            f"{examples_text}"
            "請翻譯以下內容：\n"
            f"{instruction}\n\n"
            "請輸出翻譯結果："
        )
    return prompt
```

Figure 1

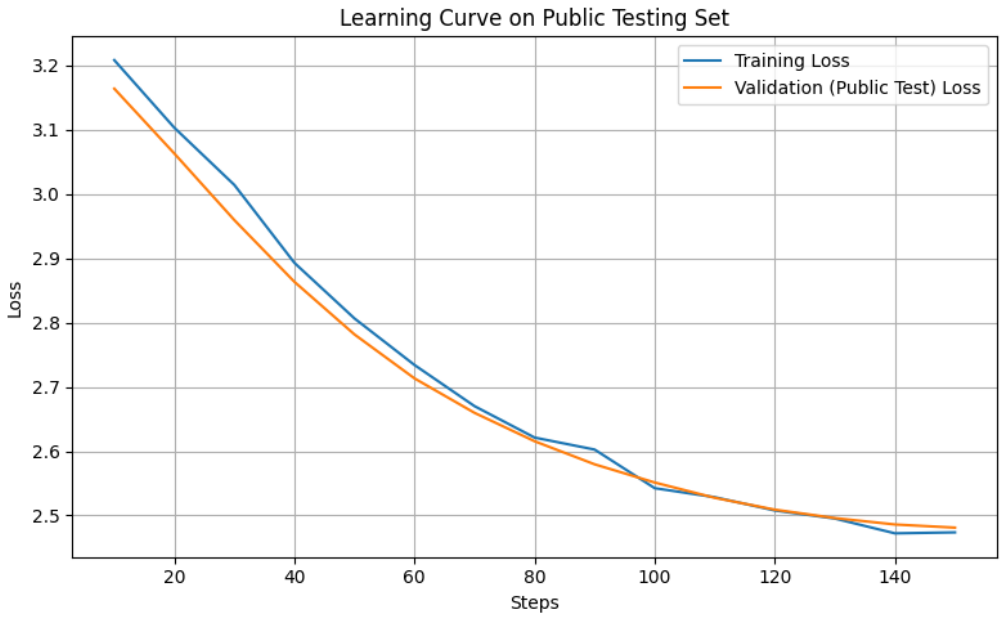

Figure 2

```python
training_args = TrainingArguments(
    output_dir=output_dir,
    overwrite_output_dir=True,
    do_train=True,
    do_eval=True,
    do_predict=True,

    # batch setting
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=16,
    bf16=True,
    gradient_checkpointing=True,

    # training setting
    num_train_epochs=1,
    learning_rate=3e-5,
    weight_decay=0.01,
    warmup_ratio=0.03,

    # validation and saving
    eval_strategy="steps",
    eval_steps=10,
    save_strategy="steps",
    save_steps=10,
    save_total_limit=2,
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
    greater_is_better=False,

    # log
    logging_strategy="steps",
    logging_steps=10,

    # CPU / dataloader
    dataloader_num_workers=12,
)
```

Figure 3

Q2: LLM Inference Strategies

Zero-Shot

The setting for zero-shot and few-shot are the same as Figure 5. For zero-shot, I design the prompt by telling the model its role and the task description as Figure 4.

Few-Shot (In-context Learning)

For few-shot, I design the prompt by randomly pick 4 examples to be in-context examples from training dataset each time, which is the same as the one used for fine-tuning the model with QLoRA as Figure 1.

Comparison

- There is more repeating part of the prompt in the results of LoRA.
- In the results of few-shot, the outputs tend to start with "\n\n好的，我需要将用户提供的文言文句子翻译成现代文。首先，我需要理解句子的结构和每个词语的意思。\n" or something similar.
- Though I use traditional Chinese prompts for all these 3 inferences, the results of few-shot show significantly higher probability to reply with simplified Chinese.
- They all seem not learning well about where to stop. The results show some obvious truncation.
- Garbled texts ❓: few-shot (6) < zero-shot (9) < LoRA (16)

```python
def get_prompt(instruction) -> str:
    prompt = (
            "角色：你是精通繁體中文的文言文與白話文雙向翻譯的大學中文系教授。\n"
            "任務：若輸入為文言文，翻譯成白話文；若輸入為白話文，翻譯為文言文。\n"
            f"{instruction}\n"
        )
    return prompt
```

Figure 4

```python
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    offload_folder=offload_dir,
    low_cpu_mem_usage=True,
    trust_remote_code=True,
    torch_dtype=torch.bfloat16,
    load_in_4bit=False
)
:
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=256,
    eos_token_id=tokenizer.eos_token_id,
    do_sample=True,
    top_k=50,
    top_p=0.95,
    temperature=0.8,
    use_cache=True,
)
```

Figure 5

Q3: Bonus: Try Llama3-Taiwan (8B)

- The settings are as Figure 6.
- The results of Llama3-Taiwan are mostly direct short answers of the translation without redundantly repeating the prompts or trying to explain the prompts.
- The results are in traditional Chinese.
- Nonetheless, there are still 11 garbled texts.

```
training_args = TrainingArguments(
    output_dir=output_dir,
    overwrite_output_dir=True,
    do_train=True,
    do_eval=True,
    do_predict=True,

    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    gradient_accumulation_steps=8,
    bf16=False,
    fp16=True,
    gradient_checkpointing=True,

    num_train_epochs=1,
    learning_rate=5e-4,
    weight_decay=0.01,
    warmup_ratio=0.03,

    eval_strategy="steps",
    eval_steps=50,
    save_strategy="steps",
    save_steps=50,
    save_total_limit=2,
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
    greater_is_better=False,

    logging_strategy="steps",
    logging_steps=50,

    dataloader_num_workers=12,
    optim="adamw_8bit",
)
```

Figure 6